The Divine Healer's Estate: An AI Game Master Agent

Stage 1 Project Design Report

Course: EECS3311

Student Name: Jiaxin Sun

Student ID: 220940359

Public GitHub repository: https://github.com/NicoleJiaxinSun/EECS3311-Divine-Healer

1. PROJECT OVERVIEW

Problem and motivation: Menu-based story games limit player expression, while unrestricted AI storytelling can contradict rules. This project combines natural-language interaction with a Java-controlled political story in a healer's estate.

Target users: Players interested in investigation, negotiation and political choices.

Agent: Interprets actions, plans multi-step goals, invokes game commands as tools, and uses action history as memory. Players approve plans; execution pauses on failure.

Model: deepseek-flash, accessed through DeepSeekAdapter.

AI interprets, plans and narrates. Java determines permissions, NPC acceptance, resources and outcomes. NPC interests are fixed; knowledge and relationships change.

Architecture: Swing GUI / CLI -> GameController -> AI components -> commands -> GameRules / GameState -> political, event and history listeners.

Four events occur in a fixed order with alternative endings. Resources and labor records support the political story. Story details and numerical values remain flexible.

2. FEATURE SPECIFICATIONS

All features are accessible through GUI and equivalent CLI commands. Natural-language access is F01; typed actions remain deterministic. Optional AI narration does not determine game results. Each entry lists input -> workflow -> output.

F01 Natural-language actions [Hybrid]

GUI: Action input. Text -> interpret, validate, execute -> action result.

Error: Ambiguous input, unsupported action or API failure prevents execution.

F02 Investigation [Deterministic]

GUI: Inspect target. Record/location -> check access, inspect -> findings/evidence.

Error: Inaccessible targets are refused; repeated discovery does not duplicate evidence.

F03 NPC negotiation [Hybrid]

GUI: NPC dialogue. Request -> apply fixed NPC strategy -> decision and dialogue.

Error: Unmet conditions yield refusal; failed narration retains the rule-based result.

F04 Evidence submission [Deterministic]

GUI: Select evidence and NPC. Known evidence -> evaluate submission -> NPC knowledge/progress.

Error: Undiscovered or irrelevant evidence is refused; duplicates do not repeat rewards.

F05 Resource allocation [Deterministic]

GUI: Resource panel. Resource, quantity, recipient -> check/deduct -> allocation and consequences.

Error: Invalid quantity, insufficient stock or authority causes refusal without deduction.

F06 Official orders [Deterministic]

GUI: Orders panel. Request/approve/execute -> check authority/status -> order or record change.

Error: An unapproved or already executed order cannot be executed.

F07 Political appointment [Deterministic]

GUI: Request appointment. Player standing and NPC request -> evaluate -> rank change or refusal.

Error: Unmet conditions grant no promotion; AI cannot grant authority.

F08 Political event progression [Deterministic]

GUI: Event panel and relevant actions. Action facts -> evaluate current event -> ending/next event.

Error: Locked/finished events accept no progress; a failed ending may unlock the next event.

F09 AI plan generation [AI-based]

GUI: Goal input. Goal and visible context -> generate ordered steps -> draft plan.

Error: Invalid plan or API failure produces no execution or world changes.

F10 Approved plan execution [Deterministic]

GUI: Review, approve, run/cancel. Approved plan -> execute validated steps -> results/status.

Error: Failure pauses execution; earlier successes remain. Retry requires approval.

F11 History and recap [Hybrid]

GUI: History panel. Recorded actions -> display and optionally summarize -> history/AI recap.

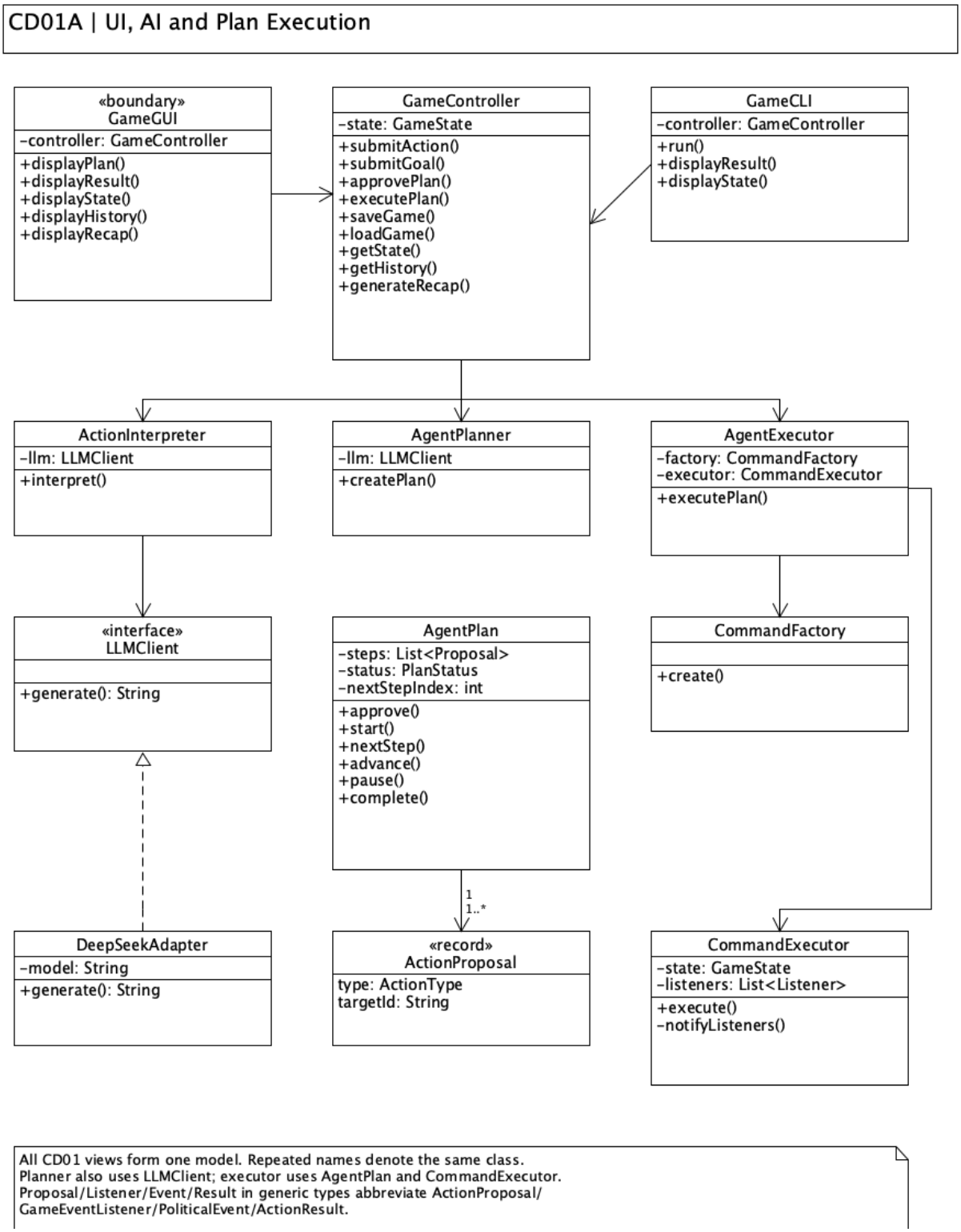
Error: Empty history shows no entries; API failure leaves structured history available.

F12 Save/load [Deterministic]

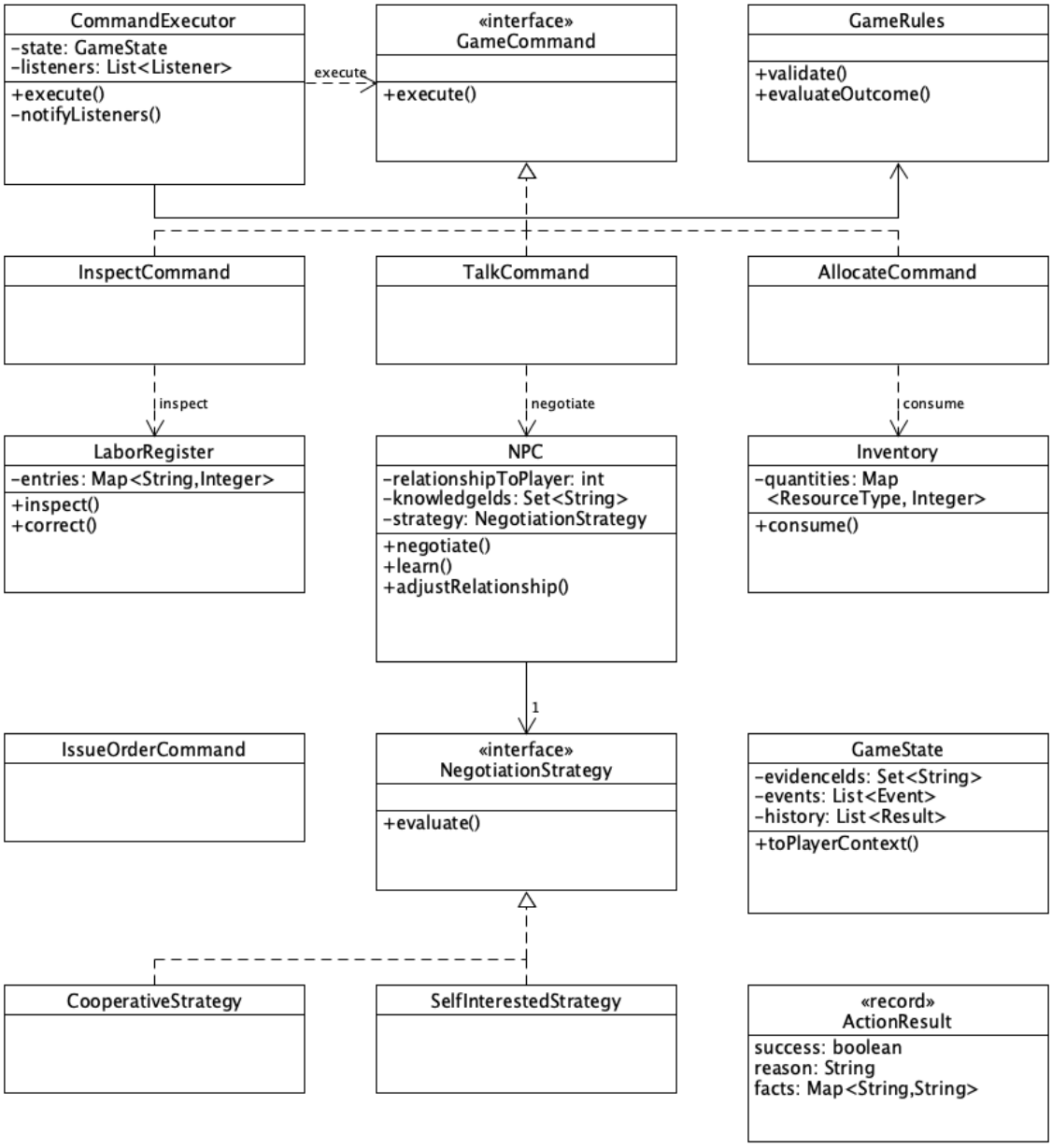
GUI: Save/Load. File path and game state -> serialize/restore -> stored or resumed game.

Error: Invalid/unreadable files preserve the current game; loading does not auto-run plans.

3. UML CLASS DIAGRAM

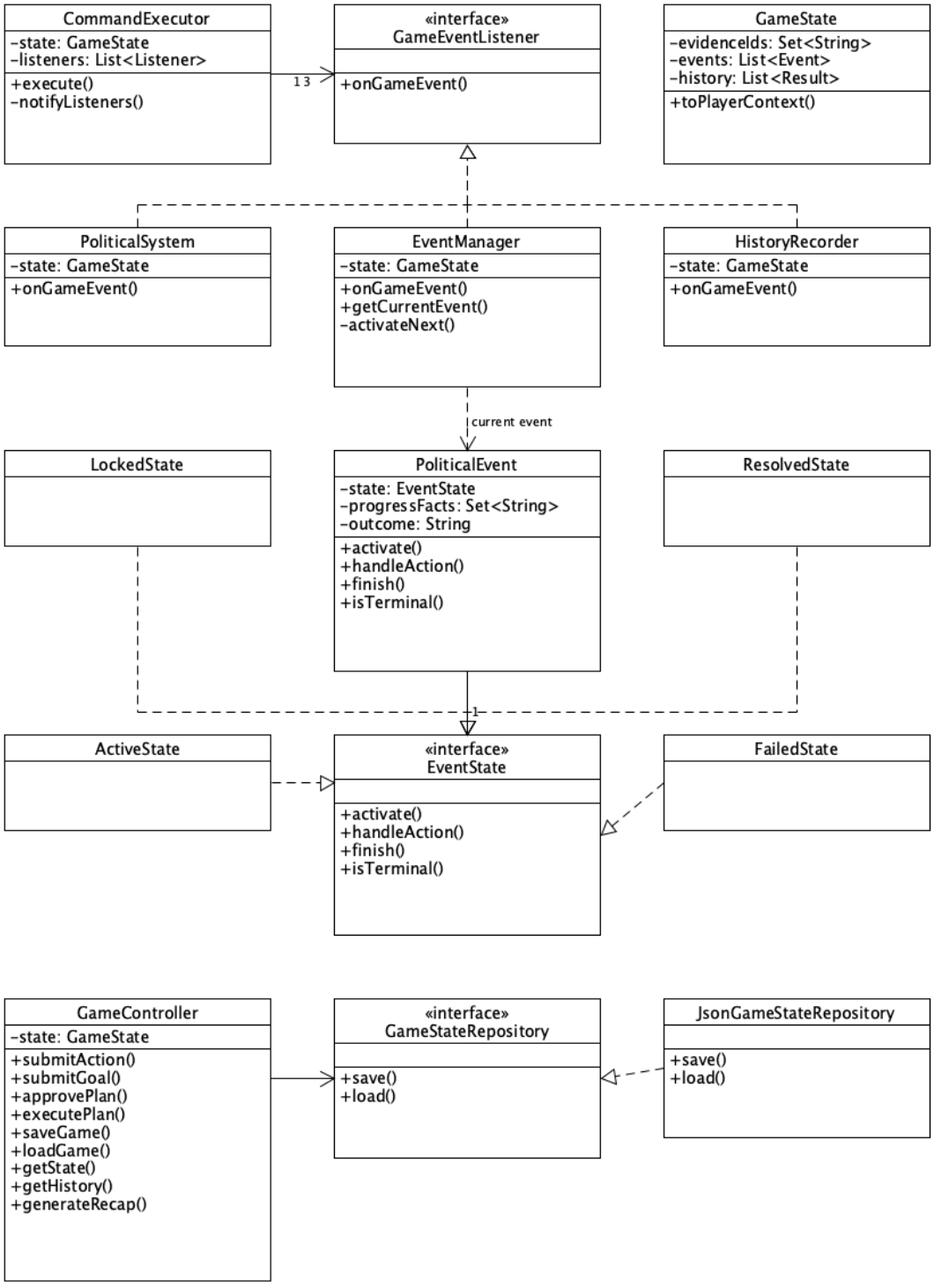


CD01B | Commands, Domain and NPC Strategy



IssueOrderCommand also implements GameCommand and uses LaborRegister.
GameState owns 1 Inventory, 1 LaborRegister and 0..* NPCs/results/plans.
It stores player rank, politics and discovered evidence. Commands use GameState.
Java decides results; NPC strategy decisions are deterministic.

CD01C | Observers, Event State and Persistence



GameState owns exactly 4 PoliticalEvents. All three listeners reference GameState.
Notification order: PoliticalSystem -> EventManager -> HistoryRecorder.
Events run E1 -> E2 -> E3 -> E4; each has one current EventState.
EventManager also uses GameRules; Controller holds the current GameState.

CD01 shows the core classes, interfaces and their relationships.

Main groups:

- UI: GameGUI, GameCLI, GameController.

- AI: LLMClient, DeepSeekAdapter, ActionInterpreter, AgentPlanner, AgentExecutor, AgentPlan.

- Execution: GameCommand, four commands, CommandFactory, CommandExecutor, GameRules.

- Domain: GameState, NPC, Inventory, LaborRegister.

- Story: PoliticalEvent, EventManager, EventState and its four concrete states.

- Consequences: GameEventListener, PoliticalSystem, HistoryRecorder.

- Persistence: GameStateRepository, JsonGameStateRepository.

Key design contracts:

GameController: submitAction(text), submitGoal(goal), approvePlan(id), executePlan(id), saveGame(), loadGame().

ActionInterpreter.interpret(); AgentPlanner.createPlan(); LLMClient.generate().

AgentExecutor.executePlan(); AgentPlan: steps, status, nextStepIndex.

CommandFactory.create(); GameCommand.execute(state); GameRules.validate().

NPC: interests, relationship, knowledge; negotiate(); NegotiationStrategy.evaluate().

Inventory: quantities; LaborRegister: entries.

PoliticalEvent: state, outcome; handleAction(), finish(); GameRules.evaluateOutcome().

GameEventListener.onGameEvent(); repository.save()/load().

Supporting data: ActionProposal (type/target/parameters), ActionResult

(success/reason/facts), NegotiationResult (accepted/reason).

GameState stores player rank, political attributes and discovered evidence, and owns Inventory, LaborRegister, four events, NPCs, plans and history.

Each NPC has one strategy; each event has one state.

4. DESIGN PATTERN EXPLANATIONS

Command: GameCommand/four commands encapsulate actions; CommandExecutor invokes

operations on domain receivers. One path serves GUI, CLI and plans, avoiding duplicated rules.

Adapter: LLMClient is the target; DeepSeekAdapter wraps the external API for AI

clients. This isolates provider changes instead of spreading API details through the game.

Strategy: NPC delegates to NegotiationStrategy; CooperativeStrategy and

SelfInterestedStrategy provide fixed decision policies, avoiding NPC-specific conditionals.

State: PoliticalEvent delegates to LockedState, ActiveState, ResolvedState or

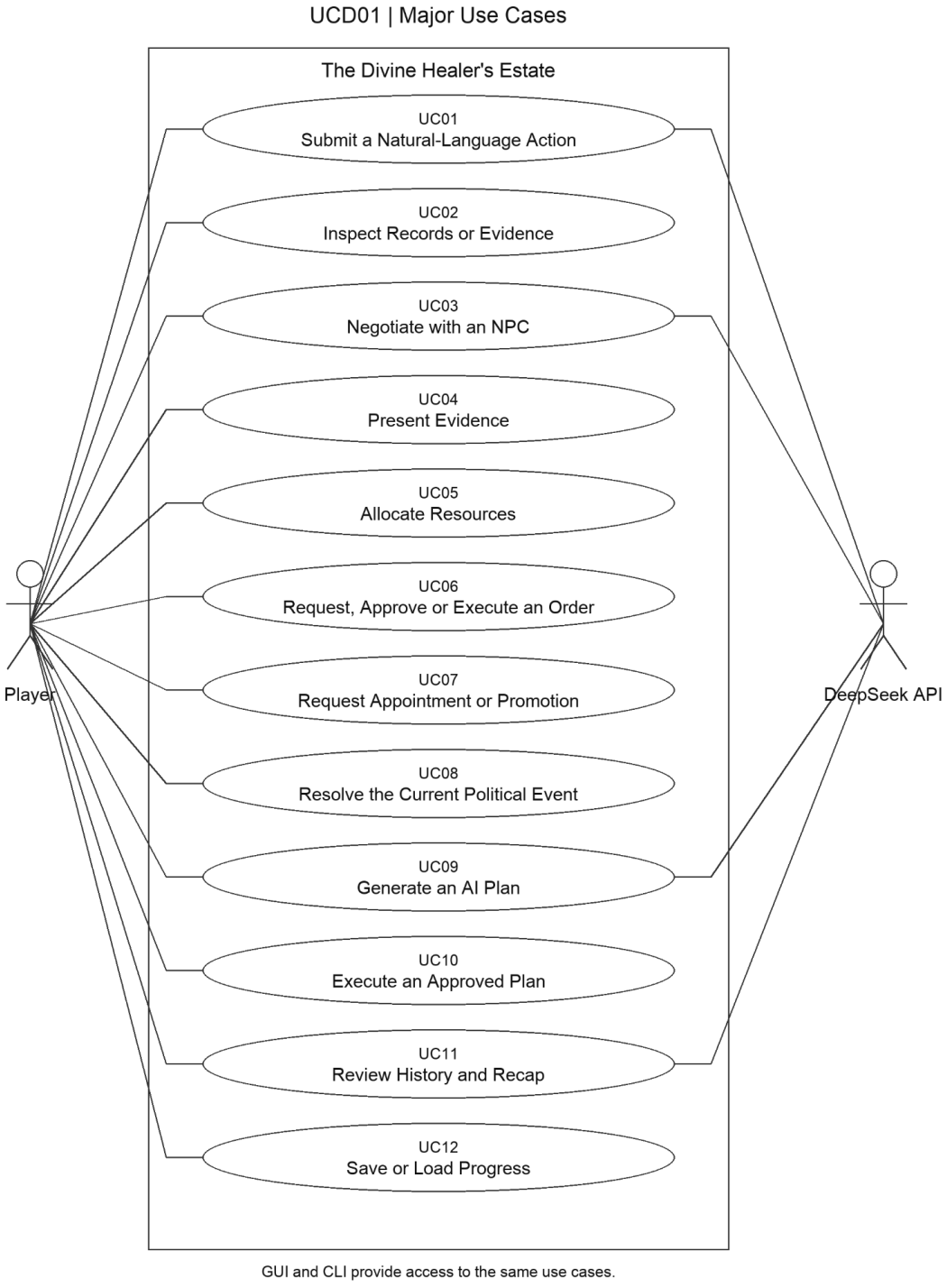
FailedState through EventState, avoiding scattered lifecycle checks and repeated endings.

Observer: CommandExecutor notifies PoliticalSystem, EventManager and HistoryRecorder

through GameEventListener, avoiding direct command dependencies on all three systems.

5. USE-CASE DIAGRAM

UCD01 - Major Use Cases



Actors: Player; external DeepSeek API. NPCs and the agent are internal components.

Player connects to UC01-UC12 below. DeepSeek supports UC01, UC03, UC09 and UC11.

GUI and CLI are alternate interfaces, not separate user actors.

6. DETAILED USE-CASE DESCRIPTIONS

Common precondition: A game is initialized and no conflicting operation is running.

Actor: Player; DeepSeek is a supporting actor where AI is involved.

Each UC corresponds to the same-numbered Feature. Failed operations grant no success effects. Each entry states goal, trigger, main flow, alternative and postcondition.

UC01 Perform natural-language action [Player, DeepSeek]

Goal: Express an intention. Trigger: Submit text. Flow: Interpret -> validate -> execute -> display.

Alternative: Ask to revise invalid input. Postcondition: Result returned; permitted effects applied.

UC02 Investigate [Player]

Goal: Obtain evidence. Trigger: Inspect target. Flow: Check access -> read -> record findings.

Alternative: Refuse inaccessible target. Postcondition: Eligible evidence becomes known.

UC03 Negotiate [Player, DeepSeek]

Goal: Obtain NPC cooperation. Trigger: Submit request. Flow: Validate -> strategy decision -> dialogue.

Alternative: Refusal or narration fallback. Postcondition: Decision and permitted consequences recorded.

UC04 Submit evidence [Player]

Precondition: Evidence is known. Goal: Support a claim. Trigger: Submit to NPC.

Flow: Check evidence -> evaluate -> update knowledge/progress.

Alternative: Reject irrelevant evidence. Postcondition: Accepted evidence is registered.

UC05 Allocate resources [Player]

Goal: Provide relief. Trigger: Confirm allocation. Flow: Check stock/authority -> deduct -> apply effects.

Alternative: Refuse invalid allocation. Postcondition: Stock and consequences reflect the allocation.

UC06 Operate official order [Player]

Goal: Carry out authorized decisions. Trigger: Request, approve or execute order.

Flow: Validate authority/status -> change order status -> apply change only on execution.

Alternative: Refuse unauthorized/repeated execution. Postcondition: Valid operation is recorded.

UC07 Request appointment [Player]

Goal: Gain authority. Trigger: Submit appointment request. Flow: Check conditions -> NPC decision -> rank update.

Alternative: Refuse unmet conditions. Postcondition: Authorized promotion applied or rank unchanged.

UC08 Resolve political event [Player]

Precondition: An event is active. Goal: Progress story. Trigger: Complete a relevant action.

Flow: Update politics/progress -> determine ending -> unlock next event.

Alternative: Failure ending; ignore unrelated facts. Postcondition: Progress or event status updated.

UC09 Generate plan [Player, DeepSeek]

Goal: Decompose a goal. Trigger: Generate Plan. Flow: Build visible context -> generate -> show draft.

Alternative: Reject invalid response. Postcondition: Reviewable plan; no actions executed.

UC10 Execute approved plan [Player]

Precondition: A draft/paused plan exists. Goal: Carry out a goal. Trigger: Approve and Run.

Flow: Confirm exact steps -> validate/execute each -> show progress.

Alternative: Failure pauses; cancellation stops later steps. Postcondition: Completed/paused/cancelled plan.

UC11 Review history/recap [Player, DeepSeek for recap]

Goal: Recall decisions. Trigger: Open History / Generate Recap. Flow: Read records -> optional summary.

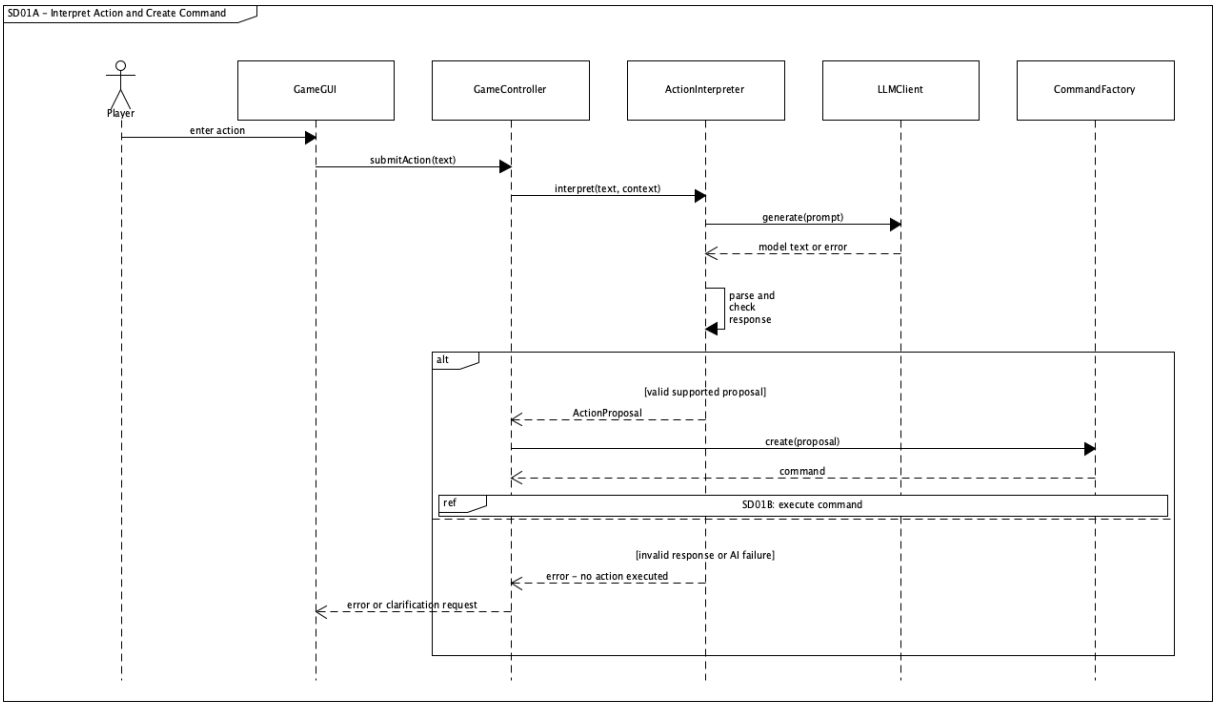
Alternative: Empty history/API failure. Postcondition: History or recap displayed; game unchanged.

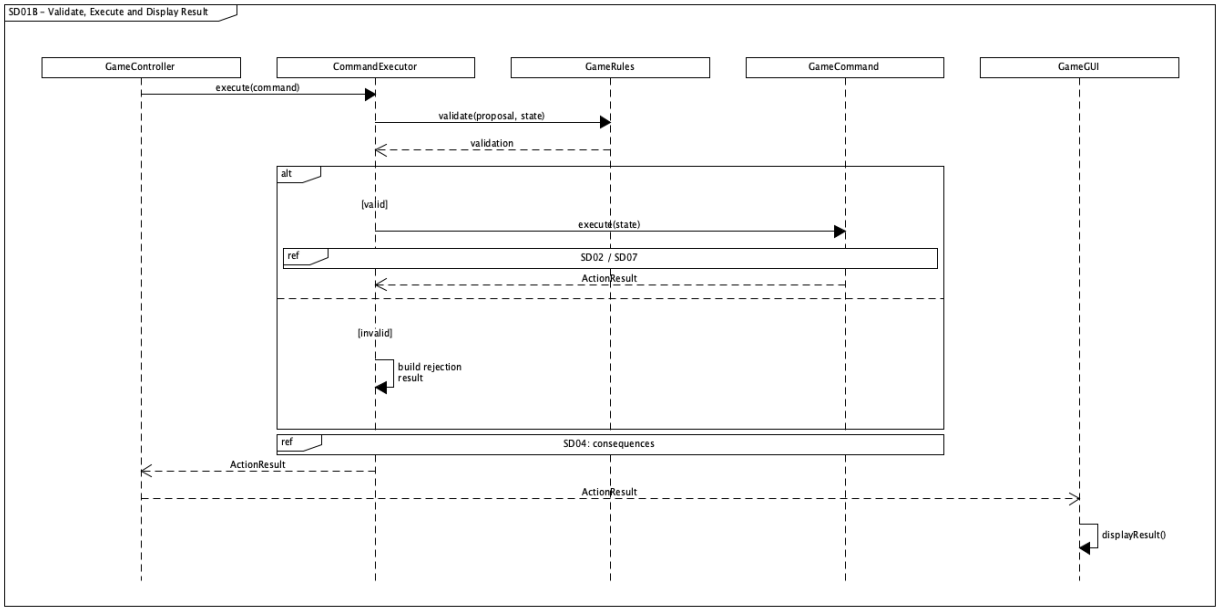
UC12 Save/load game [Player]

Goal: Resume progress. Trigger: Save/Load. Flow: Serialize state or validate/restore file -> refresh.

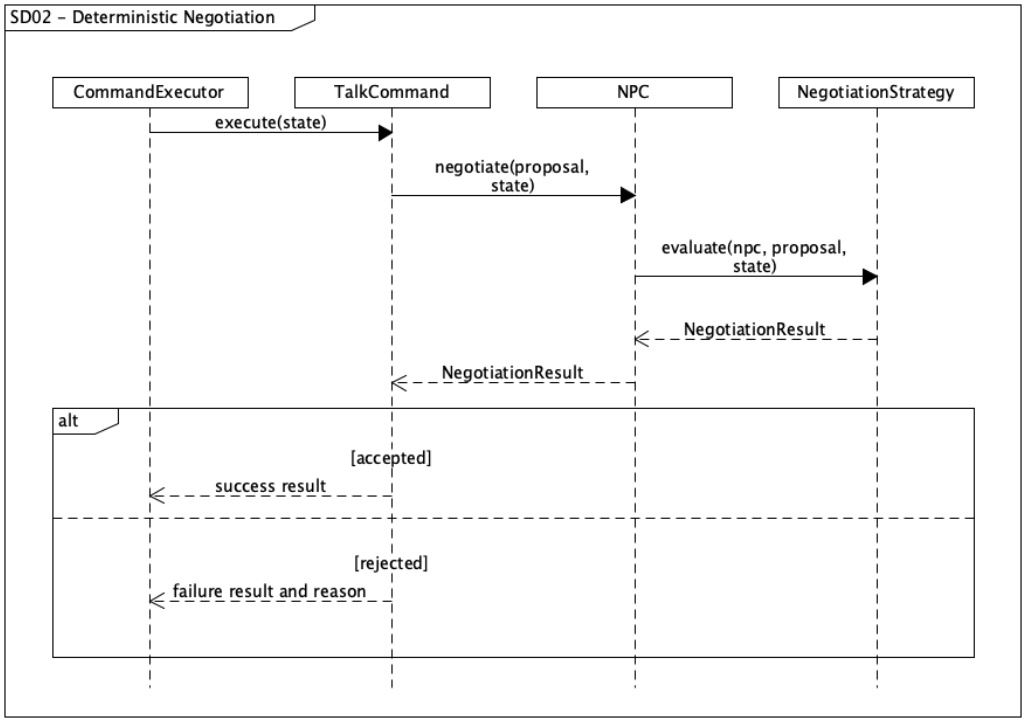
Alternative: Report file error without replacing current game. Postcondition: Saved/restored progress.

7. SEQUENCE DIAGRAMS

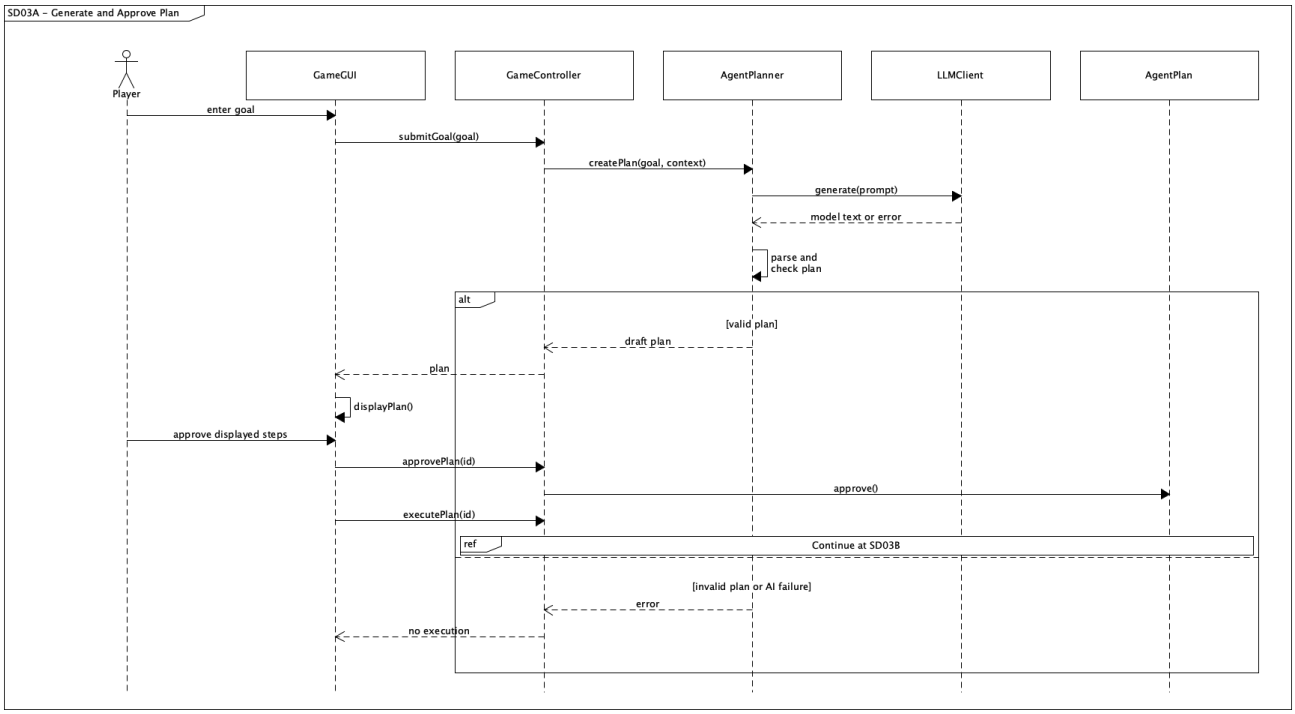


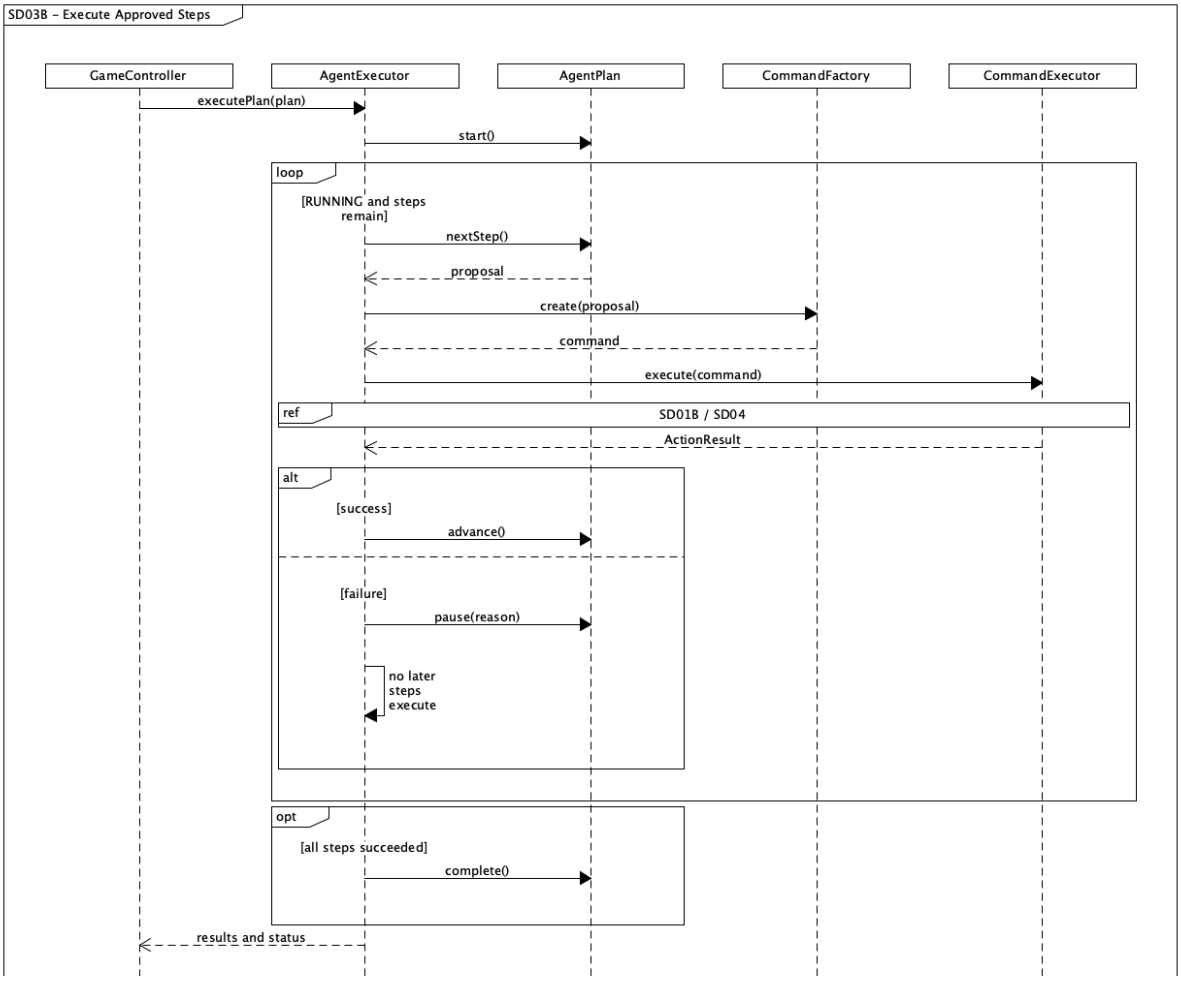


SD01 shows action interpretation, command creation, rule validation, execution and result notifications.

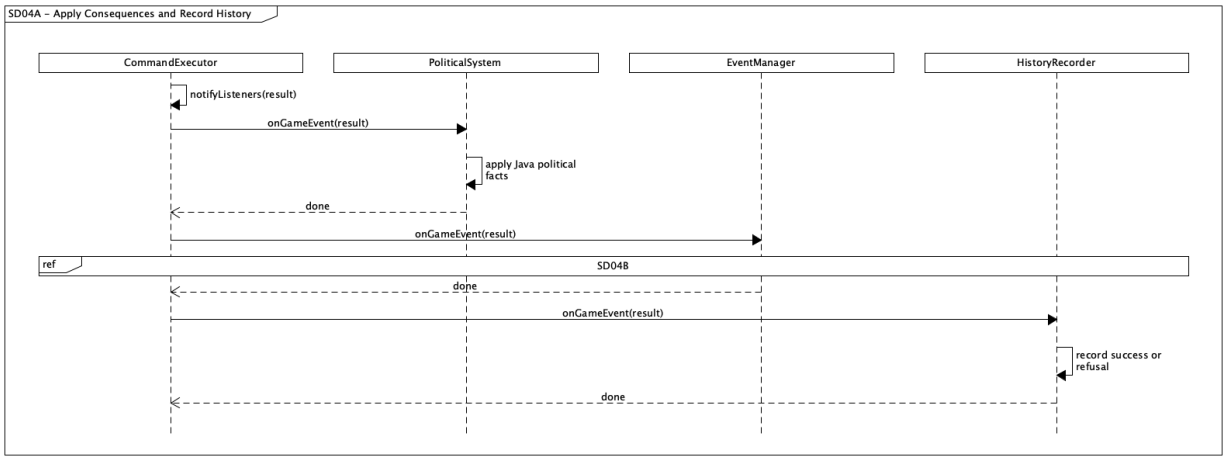


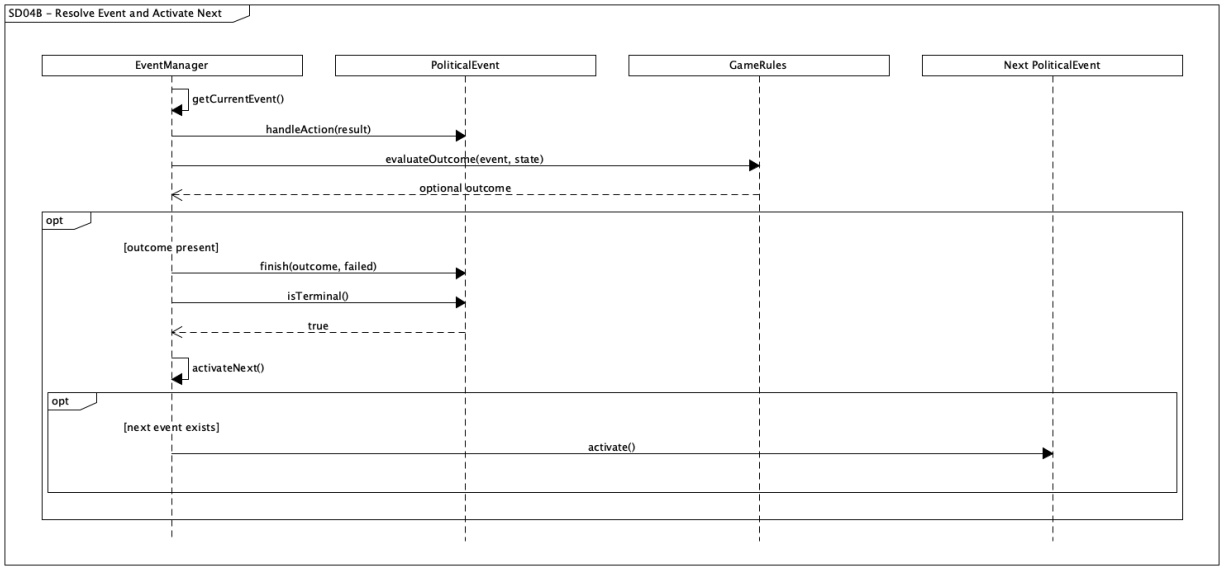
SD02 shows NPC negotiation through NegotiationStrategy and the resulting acceptance or refusal.



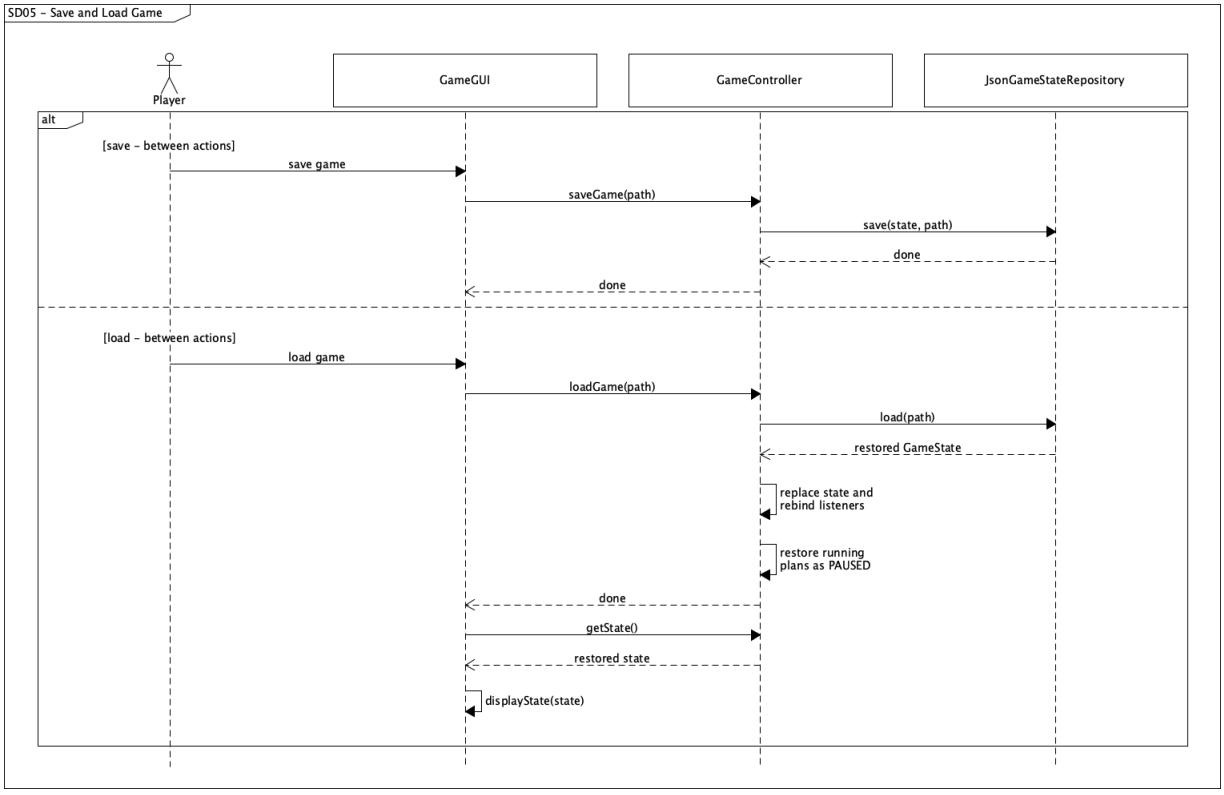


SD03 shows AI plan generation, player approval and step-by-step execution through AgentExecutor.

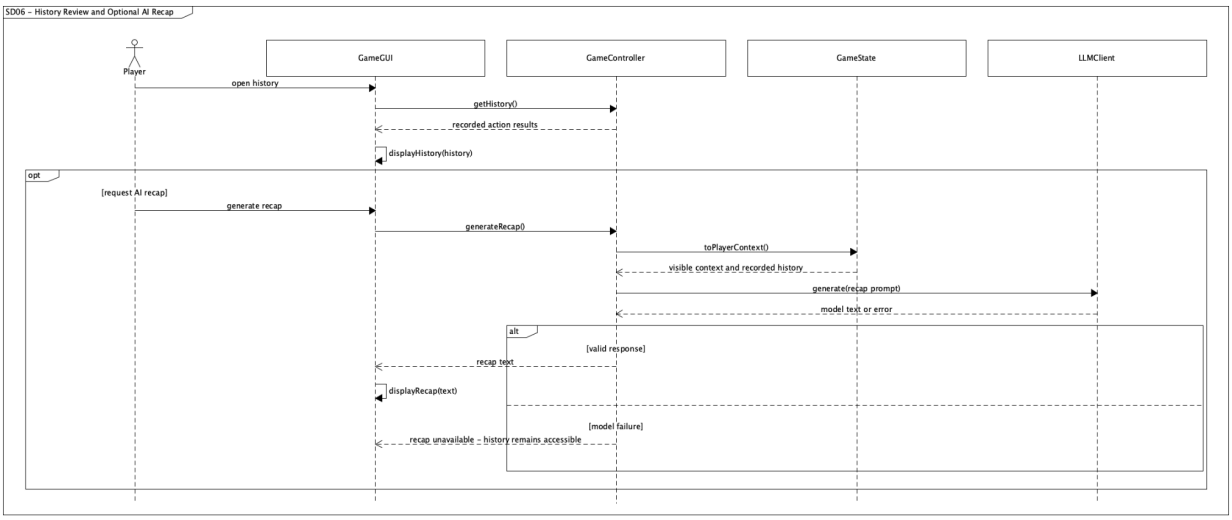




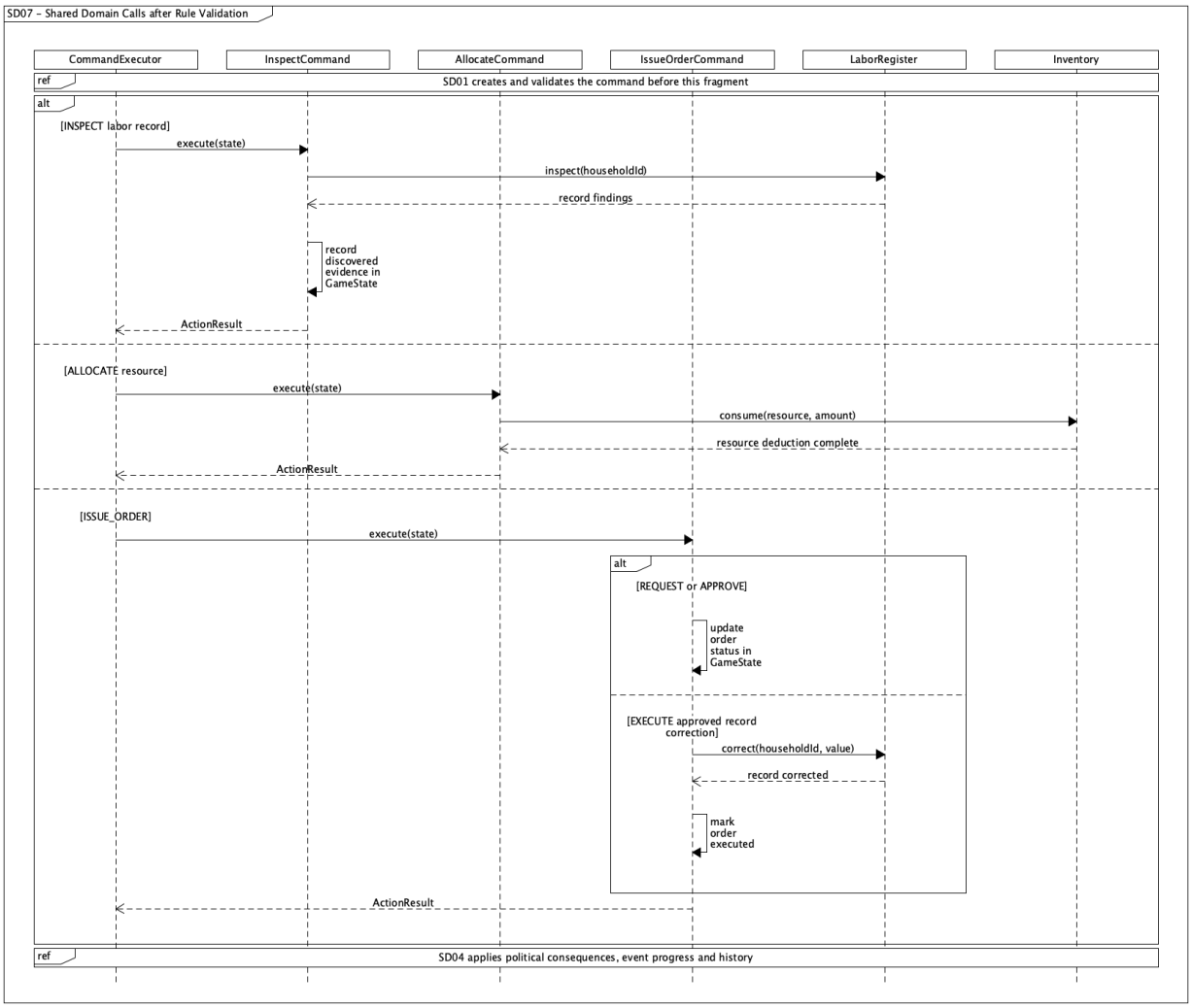
SD04 shows political consequences, event progression and history recording.



SD05 shows game-state saving and loading through the repository.



SD06 shows History and AI Recap



SD07 shows Command Domain Calls

8. FEATURE-TO-DESIGN TRACEABILITY TABLE

Feature	Use case	Main classes	Key methods	Sequence	Patterns
F01	UC01	Controller, Interpreter, LLMClient, Factory, Executor	submitAction, interpret, generate, create, execute	SD01	Command, Adapter
F02	UC02	InspectCommand, LaborRegister, GameState	execute, inspect	SD01, SD04, SD07	Command, Observer
F03	UC03	TalkCommand, NPC, NegotiationStrategy	execute, negotiate, evaluate	SD01, SD02	Command, Strategy, Adapter
F04	UC04	TalkCommand, NPC, EventManager	execute, learn, onGameEvent	SD01, SD02, SD04	Strategy, Observer
F05	UC05	AllocateCommand, Inventory	execute, consume	SD01, SD04, SD07	Command, Observer
F06	UC06	IssueOrderCommand, GameRules, LaborRegister	validate, execute, correct	SD01, SD04, SD07	Command, Observer
F07	UC07	TalkCommand, NPC, PoliticalSystem, GameState	negotiate, onGameEvent	SD02, SD04	Strategy, Observer
F08	UC08	EventManager, PoliticalEvent, EventState, GameRules	onGameEvent, handleAction, evaluateOutcome, finish	SD04	Observer, State
F09	UC09	Controller, AgentPlanner, LLMClient, AgentPlan	submitGoal, createPlan, generate	SD03	Adapter
F10	UC10	GameController, AgentExecutor, AgentPlan, Factory, Executor	approvePlan, executePlan, create, execute	SD03	Command
F11	UC11	HistoryRecorder, Controller, LLMClient	onGameEvent, getHistory, generateRecap, generate	SD04 (history recording), SD06	Observer, Adapter
F12	UC12	Controller, GameStateRepository, GameState	saveGame, loadGame, save, load	SD05	Persistence support; no additional pattern claimed

Controller/Factory/Executor abbreviate GameController/CommandFactory/CommandExecutor.

The feature classifications are listed in Section 2.

9. FEATURE IMPLEMENTATION EXPLANATIONS

F01: Interpreter converts text to a proposal; Factory creates a supported command;

Executor checks rules and executes it. Adapter supplies the AI call.

F02: InspectCommand reads permitted targets and records discovered evidence.

F03: TalkCommand delegates acceptance to NPC strategy; AI describes the fixed decision.

F04: Accepted evidence updates NPC knowledge and supplies facts to event listeners.

F05: AllocateCommand deducts valid resources; listeners apply political/story effects.

F06: IssueOrderCommand separates request/approval from authorized execution.

F07: PoliticalSystem applies a rule-approved appointment to the player rank stored in GameState.

F08: EventManager delegates progress to the active event and evaluates its ending.

F09: Planner uses visible state/history to create a draft without executing actions.

F10: AgentExecutor invokes commands step by step and pauses immediately on failure.

F11: HistoryRecorder stores outcomes; Controller displays records or generates a recap.

F12: Repository saves/restores domain data; Controller reconnects services to loaded state.